

Université de Lorraine

IUT Nancy Charlemagne

Département Informatique

Rapport Final

SAE 3.01

Développement d'une Application

Application de Gestion de Projet

Type Trello – JavaFX

Auteurs :

Nathan SPÄTH

Léo BOUGNOUX

Gabriel MARIQUE

Année universitaire :

2025 – 2026

Date de rendu :

Janvier 2026

Tuteurs :

Mme DEBLED-RENNESSON

M. BINET

Table des matières

1	Introduction	2
2	Fonctionnalités Réalisées	2
2.1	Itération 1 – Gestion de Base des Tâches	2
2.2	Itération 2 – Enrichissement des Tâches	2
2.3	Itération 3 – Vues et Archivage	2
2.4	Itération 4 – Gestion des Listes et Projets	2
2.5	Itération 5 – Étiquettes et Filtres (En cours)	3
2.6	Itération 6 – Fonctionnalités Complémentaires (À venir)	3
3	Diagrammes de Classe	4
3.1	Diagramme de Classe Complet	4
3.2	Diagramme de Classe MVC	4
3.3	Diagramme de Classe Simplifié	5
4	Répartition du Travail	6
4.1	Nathan Späth	6
4.2	Léo Bougnoux	6
4.3	Gabriel Marique	7
5	Éléments Originaux	8
5.1	Nathan Späth	8
5.2	Léo Bougnoux	9
5.3	Gabriel Marique	10
6	Modifications par Rapport à l’Analyse Préalable	12
7	Patrons de Conception et Architecture	12
7.1	Patron Composite	12
7.2	Patron MVC (Model-View-Controller)	12
7.3	Patron Singleton	12
7.4	Patron Observateur	12
8	Graphe de Scène et Organisation des Vues	13
8.1	Diagramme de Scène Principal	13
8.2	Diagramme de Scène – Vue Kanban	13
8.3	Diagramme de Scène – Vue Liste	14
8.4	Diagramme de Scène – Vue Gantt	14
8.5	Diagramme de Scène – Vue Archive	15
8.6	Diagramme de Scène – Vue Calendrier	16
8.7	Organisation des Vues	16
8.8	Mécanisme de Changement de Vue	16
9	Mode d’Emploi – Lancement sur IntelliJ	17
10	Conclusion	17
11	Liens et Ressources	17

1 Introduction

Ce rapport présente le bilan du travail réalisé dans le cadre de la SAE 3.01 portant sur le développement d’une application JavaFX. Notre projet consiste en une application de gestion de projet inspirée de Trello, permettant aux utilisateurs de créer et gérer des projets, des listes de tâches et des tâches avec diverses fonctionnalités avancées.

L’application a été développée de manière itérative sur six itérations, chacune apportant un ensemble de fonctionnalités nouvelles. Ce document détaille les fonctionnalités réalisées, l’architecture mise en place, les modifications apportées par rapport à l’analyse préalable, ainsi que les contributions individuelles de chaque membre de l’équipe.

2 Fonctionnalités Réalisées

2.1 Itération 1 – Gestion de Base des Tâches

La première itération a posé les fondations de l’application avec les opérations CRUD sur les tâches :

- Création d’une tâche
- Suppression d’une tâche
- Modification d’une tâche
- Lecture d’une tâche
- Existence de sous-tâches (implémentation du patron Composite)
- Mise en place de la structure de code

2.2 Itération 2 – Enrichissement des Tâches

Cette itération a permis d’enrichir les fonctionnalités liées aux tâches :

- Dépendance d’une tâche (continuation du patron Composite)
- Affichage détaillé d’une tâche
- Gestion des dates (début et fin)
- Stockage des données par sérialisation

2.3 Itération 3 – Vues et Archivage

L’itération 3 a introduit les différentes vues de l’application :

- Vue Gantt – visualisation temporelle des tâches
- Vue Bureau (Liste) – affichage en colonnes style Kanban
- Vue Archive – consultation des tâches archivées

2.4 Itération 4 – Gestion des Listes et Projets

Cette itération a structuré la hiérarchie projet/liste/tâche :

- Création de liste de tâches

- Suppression de liste de tâches
- Modification de liste de tâches
- Création de projet
- Suppression de projet
- Modification de projet
- Correction de bugs sur l’affichage détaillé des dépendances

2.5 Itération 5 – Étiquettes et Filtres (En cours)

L’itération 5, actuellement en cours de finalisation, comprend :

- Étiquettes pour les tâches (assignation de membres)
- Filtres et recherche
- Progression des tâches (indicateurs visuels)
- Ajout de liens pour les dépendances dans la vue Gantt
- Suppression définitive des tâches dans les archives

2.6 Itération 6 – Fonctionnalités Complémentaires (À venir)

L’itération 6 est planifiée pour inclure :

- Journal de Log
- Export du diagramme de Gantt en PDF/JPEG/PNG
- Calendrier général des tâches

3 Diagrammes de Classe

Afin de présenter l'architecture de notre application de manière progressive et compréhensible, nous avons choisi de présenter les diagrammes de classe dans un ordre allant du plus détaillé au plus simplifié. Cette approche permet au lecteur de d'abord appréhender la structure complète du projet, puis de se concentrer sur les aspects architecturaux clés (MVC), et enfin de disposer d'une vue d'ensemble simplifiée pour une compréhension rapide.

3.1 Diagramme de Classe Complet

Ce diagramme présente l'intégralité des classes de l'application avec tous leurs attributs, méthodes et relations. Il offre une vision exhaustive de la structure du projet.

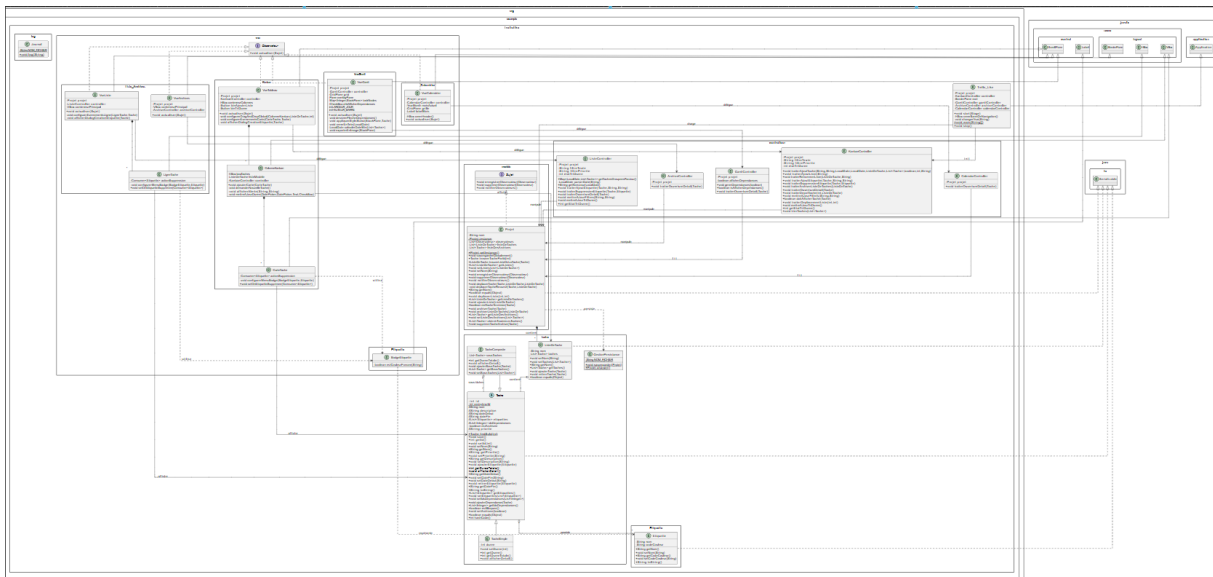


FIGURE 1 – Diagramme de classe complet de l'application

3.2 Diagramme de Classe MVC

Ce diagramme met en évidence l'architecture MVC (Modèle-Vue-Contrôleur) de l'application, en montrant les interactions entre les différentes couches.

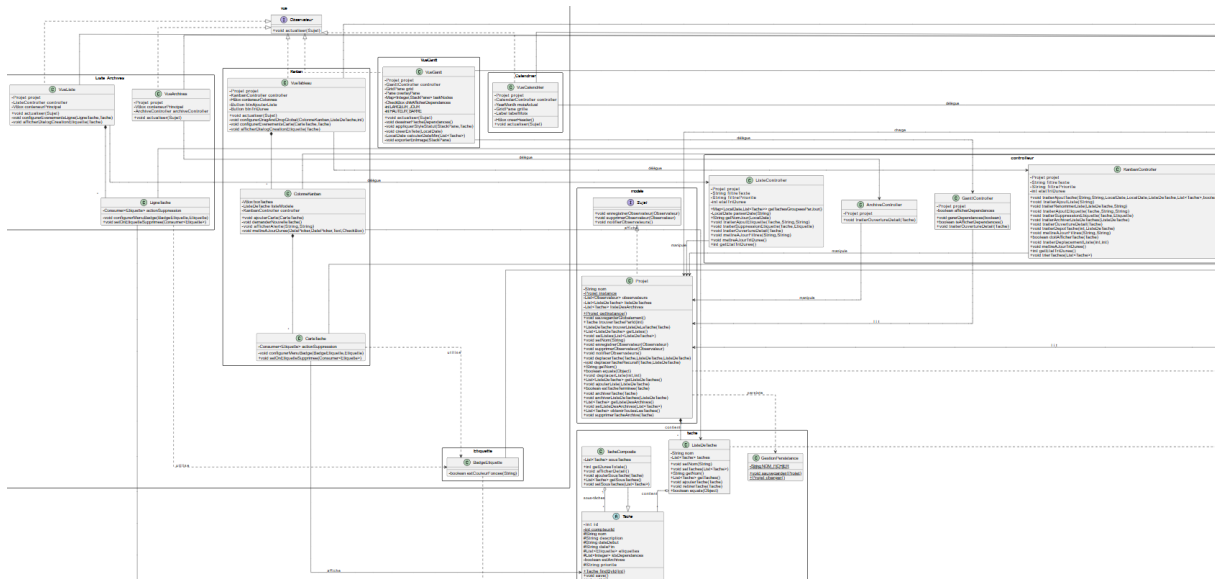


FIGURE 2 – Diagramme de classe illustrant l'architecture MVC

3.3 Diagramme de Classe Simplifié

Ce diagramme offre une vue synthétique de la structure de l'application, idéale pour comprendre rapidement les principales entités et leurs relations.

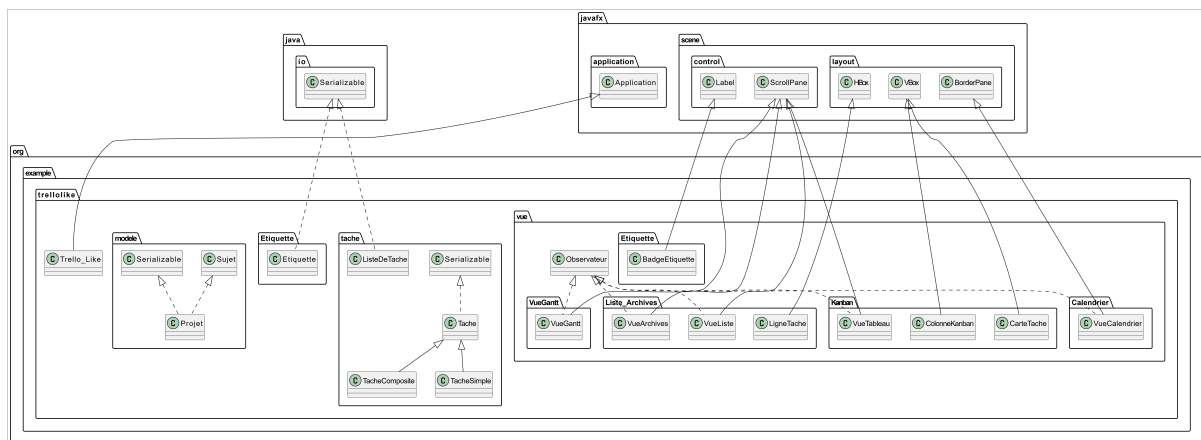


FIGURE 3 – Diagramme de classe simplifié

4 Répartition du Travail

La répartition du travail au sein de l'équipe a été gérée via le tableau Trello du projet, où chaque tâche est assignée à un ou plusieurs membres via le système d'étiquettes.

4.1 Nathan Späth

Étude Préalable :

- Faire liste cas d'utilisation + scénario si besoin (avec Gabriel)

Itération 1 :

- Existence de sous-tâche (implémentation composite)
- Création de Tâches (avec Léo)
- Lecture d'une tâche (avec Léo)
- Suppression de tâches (avec Léo)
- Structure Code (avec Léo et Gabriel)

Itération 2 :

- Affichage détaillé d'une tâche
- Modification de tâches
- Gestion des dates (avec Léo et Gabriel)
- Dépendances d'une tâche (avec Léo et Gabriel)

Itération 3 :

- Affichage détaillé tâche (dépendance) (réglage bug)

Itération 4 :

- Vue Gantt

Itération 5 :

- Suppression tâche totalement dans Archive
- Ajout lien pour dépendance dans Gantt
- Filtres et recherche

Itération 6 :

- Journal de Log
- Exporter Gantt en PDF/JPEG/...

4.2 Léo Bougnoux

Étude Préalable :

- Faire fichier compte rendu partagé
- Diagramme d'état de l'objet central de l'application
- Maquette de l'application
- Diagramme d'activité de l'application
- Liste fonctionnalité à faire (avec Gabriel)

- Planning des 6 itérations (avec Gabriel)

Itération 1 :

- Création de Tâches (avec Nathan)
- Lecture d'une tâche (avec Nathan)
- Suppression de tâches (avec Nathan)
- Structure Code (avec Nathan et Gabriel)

Itération 2 :

- Gestion des dates (avec Nathan et Gabriel)
- Dépendances d'une tâche (avec Nathan et Gabriel)

Itération 3 :

- Correction de bug

Itération 4 :

- Vue Liste

Itération 5 :

- Étiquette pour tâche

Itération 6 :

- Gérer déplacement Liste de tâches
- Gérer déplacement tâches enfants avec mère

4.3 Gabriel Marique

Étude Préalable :

- Liste fonctionnalité à faire (avec Léo)
- Planning des 6 itérations (avec Léo)
- Faire liste cas d'utilisation + scénario si besoin (avec Nathan)

Itération 1 :

- Structure Code (avec Nathan et Léo)

Itération 2 :

- Gestion des dates (avec Nathan et Léo)
- Dépendances d'une tâche (avec Nathan et Léo)

Itération 3 :

- Création liste de tâche
- Modifier liste de tâche
- Supprimer liste de tâche

Itération 4 :

- Vue Archive

Itération 5 :

- Début Rédaction du rapport

Itération 6 :

- Calendrier général des tâches

5 Éléments Originaux

Chaque membre de l'équipe présente ci-dessous un élément original du projet dont il est particulièrement fier.

5.1 Nathan Späth

L'une des fonctionnalités les plus abouties de cette application est le système de recherche et de tri en temps réel intégré aux vues Liste et Kanban. Mon objectif principal était de garantir une expérience utilisateur fluide, où l'interface réagit instantanément à la saisie sans aucun temps de chargement.

Pour assurer la cohérence de l'application, j'ai centralisé la logique de filtrage dans les contrôleurs (ListeController et KanbanController).

Réactivité instantanée : Grâce à l'utilisation d'écouteurs (listeners) sur les champs de saisie, le système détecte chaque lettre tapée et met à jour l'affichage immédiatement.

Logique de tri unique : J'ai créé une méthode spécifique (doitAfficherTache) qui sert de « filtre unique ». Elle vérifie simultanément si le nom, la description ou la priorité de la tâche correspondent aux critères de l'utilisateur.

Tri par durée flexible : Un système de tri cyclique (Neutre, Croissant, Décroissant) permet d'organiser les tâches par charge de travail. Au clic sur le bouton de tri, la vue est reconstruite pour refléter le nouvel ordre.

L'affichage a été optimisé selon le mode de lecture choisi par l'utilisateur :

En Vue Liste : Les tâches sont filtrées puis regroupées automatiquement par date via une structure de données triée (TreeMap), assurant une lecture chronologique parfaite.

En Vue Kanban : Le système filtre les tâches à l'intérieur de chaque colonne, puis recrée dynamiquement les cartes graphiques dans l'ordre de tri sélectionné.

Je suis particulièrement fier de la stabilité de cette fonctionnalité. Un soin particulier a été apporté à la gestion des erreurs : le code intègre des sécurités pour gérer les données manquantes (comme une tâche sans priorité ou sans description), évitant ainsi tout plantage.

Cette réalisation transforme une simple gestion de données en une interface « vivante » et réactive, ce qui représente, selon moi, une réelle réussite en termes d'ergonomie logicielle.

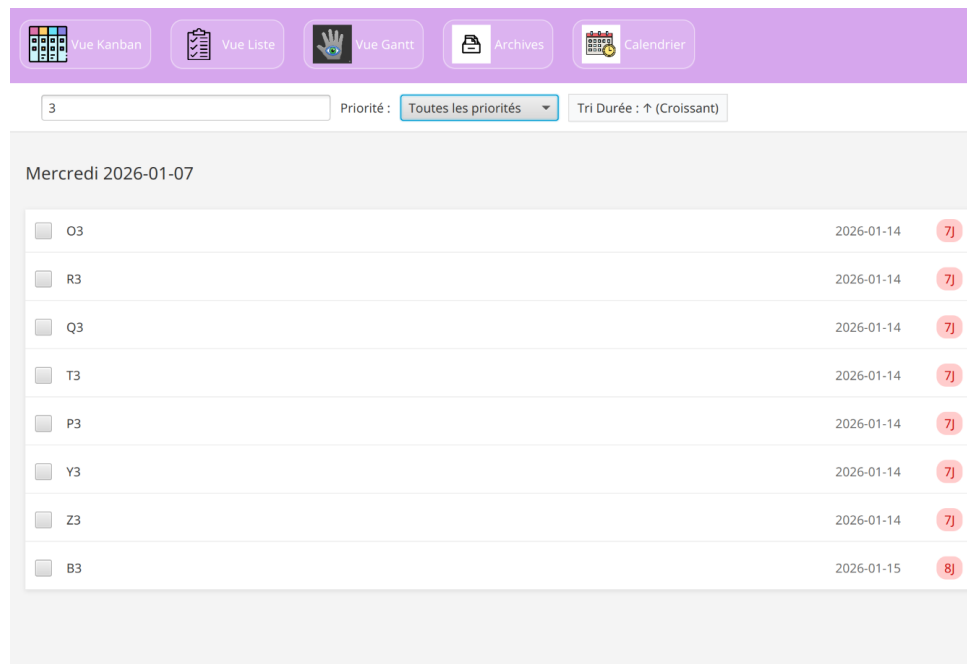


FIGURE 4 – Illustration du système de recherche et tri

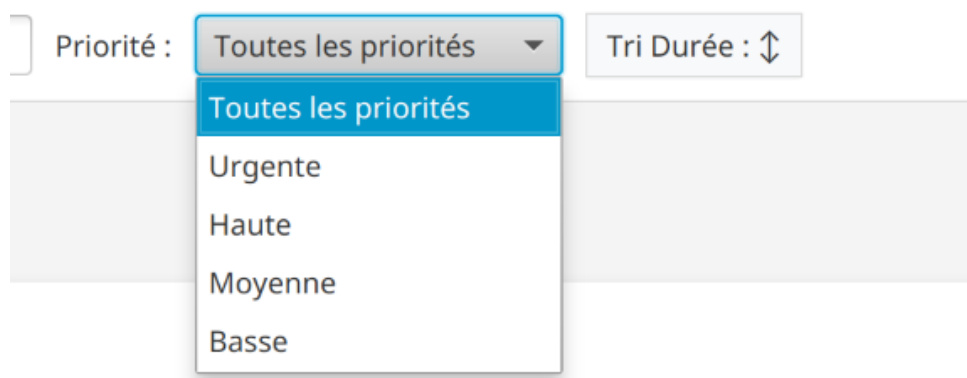


FIGURE 5 – Illustration du système de recherche et tri (suite)

5.2 Léo Bougnoux

Je suis fier de cette partie de développement de projet, les étiquettes sont très esthétiques et font partie intégrante du décor visuel de notre application Trello-Like. Au niveau du développement je me suis basé en modifiant un petit peu sur la classe étiquette créée par Nathan. Ensuite j'ai mis au point un visuel de ces dernières adaptées aux différentes vues de l'application, ajoutées aux tâches une liste d'étiquettes permettant de les stocker sans oublier les fonctions nécessaires pour la sérialisation XML c'est à dire getter et setter pour chaque classe. Enfin j'ai adapté les vues pour positionner la liste d'étiquettes au niveau des tâches ce qui amène à ce beau visuel ci-dessous.

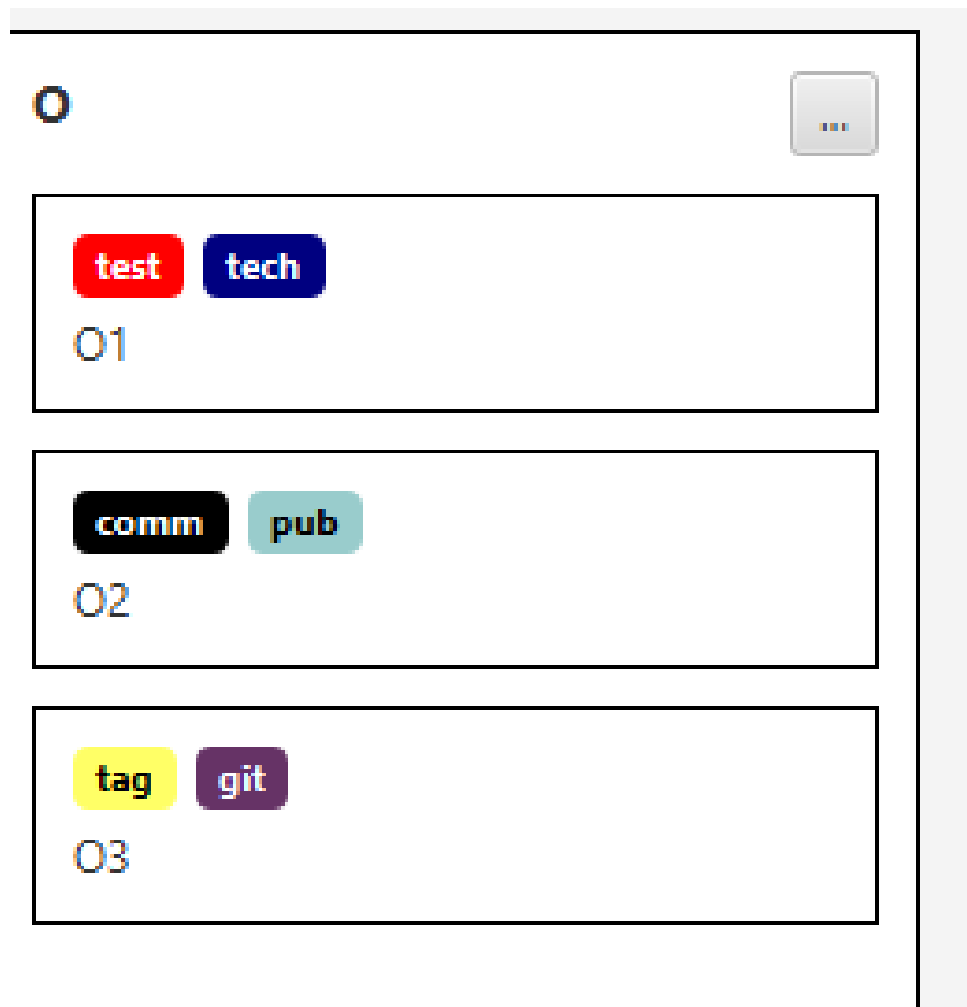


FIGURE 6 – Illustration du système d'étiquettes

5.3 Gabriel Marique

La fonctionnalité dont je suis le plus fier est le système d'archivage des tâches. Cette fonctionnalité permet aux utilisateurs de conserver un historique de leurs tâches terminées sans encombrer les vues principales (Kanban, Liste, Gantt).

Principe de fonctionnement : Lorsqu'une tâche est archivée, elle est retirée de sa liste d'origine et transférée vers une collection dédiée (listeDesArchives) dans le modèle Projet. L'utilisateur peut également archiver une liste complète en une seule action, ce qui archive automatiquement toutes les tâches qu'elle contient.

Une vue dédiée (VueArchives) affiche l'ensemble des tâches archivées et permet de consulter leurs détails ou de les supprimer définitivement. Grâce au pattern Observer, cette vue se synchronise automatiquement avec les autres vues lors de chaque modification.

Pourquoi cet élément est original : Cette fonctionnalité apporte une vraie valeur ajoutée à l'application en offrant une gestion complète du cycle de vie des tâches, de leur création jusqu'à leur archivage, tout en respectant l'architecture MVC et les patterns mis en place dans le projet.

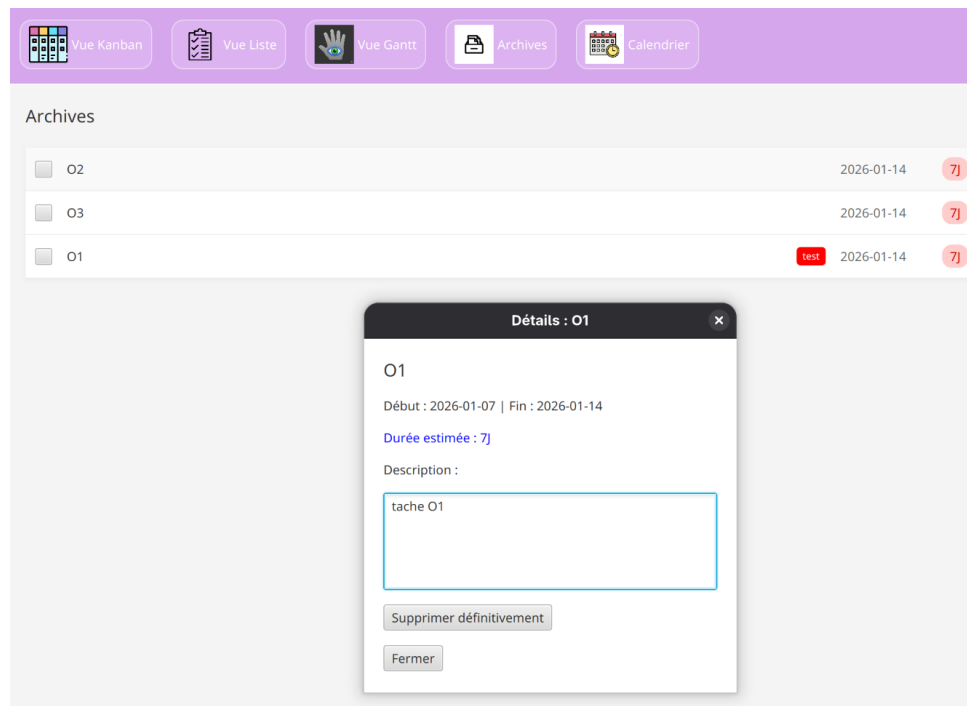


FIGURE 7 – Illustration du système d'archivage

6 Modifications par Rapport à l'Analyse Préalable

L'analyse préalable avait prévu plusieurs fonctionnalités liées à une utilisation multi-utilisateurs de l'application. Conformément aux instructions reçues précisant que l'application devait être une application JavaFX locale et non une application réseau, les fonctionnalités suivantes ont été retirées du périmètre :

- **Gestion multi-utilisateurs** – L'application fonctionne en mode mono-utilisateur local
- **Chat de groupe** – Fonctionnalité réseau non applicable en local
- **Chef de projet et modération des rôles** – Sans multi-utilisateurs, la gestion des rôles n'est plus pertinente

Ces modifications n'impactent pas les fonctionnalités principales de gestion de projet, qui restent pleinement opérationnelles.

7 Patrons de Conception et Architecture

L'application met en œuvre plusieurs patrons de conception reconnus :

7.1 Patron Composite

Le patron Composite a été utilisé pour modéliser la hiérarchie des tâches et sous-tâches. Ce patron permet de traiter uniformément les tâches simples et les tâches composées (contenant des sous-tâches), facilitant ainsi les opérations récursives sur l'arborescence.

7.2 Patron MVC (Model-View-Controller)

L'architecture générale de l'application suit le patron MVC :

- **Model** – Classes représentant les données (Projet, ListeDeTache, Tache, Etiquette)
- **View** – Interfaces JavaFX (FXML et composants graphiques)
- **Controller** – Classes contrôleurs gérant les interactions utilisateur

7.3 Patron Singleton

Le patron Singleton a été utilisé pour garantir l'existence d'une seule instance du projet au sein de l'application. Ce patron assure un point d'accès global à cette instance unique et évite les problèmes de cohérence des données.

7.4 Patron Observateur

Le patron Observateur a été implémenté pour gérer la mise à jour automatique des vues lorsque les données du modèle changent. Ce mécanisme permet de maintenir la synchronisation entre le modèle et les différentes vues de l'application.

8 Graphe de Scène et Organisation des Vues

Cette section présente les diagrammes de scène de chaque vue de l'application, illustrant la structure des composants graphiques JavaFX.

8.1 Diagramme de Scène Principal

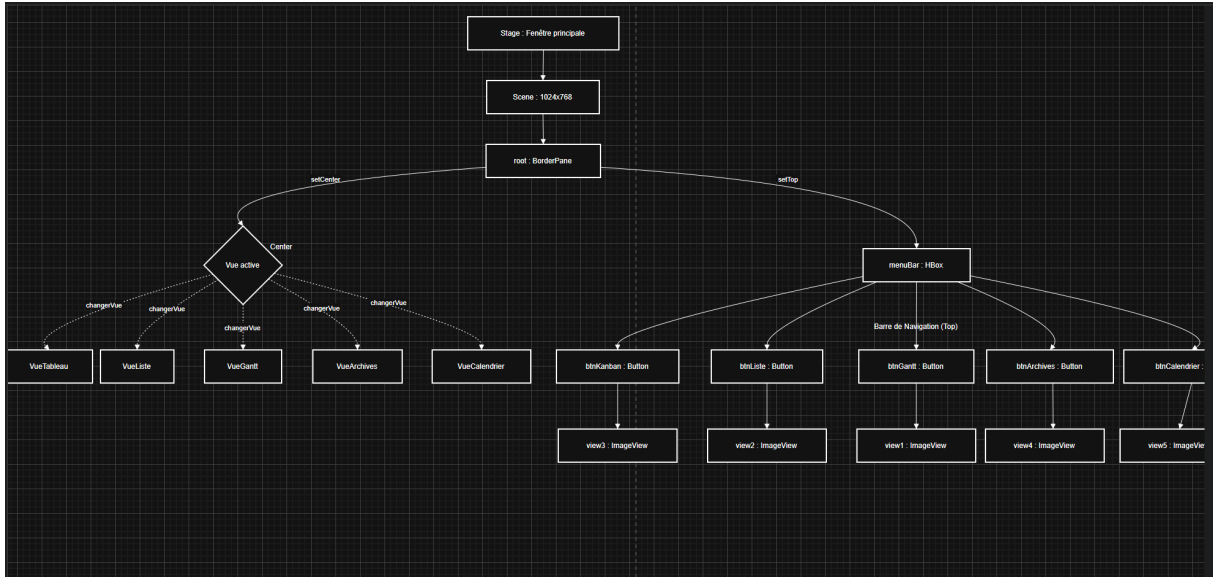


FIGURE 8 – Diagramme de scène de la vue principale

8.2 Diagramme de Scène – Vue Kanban

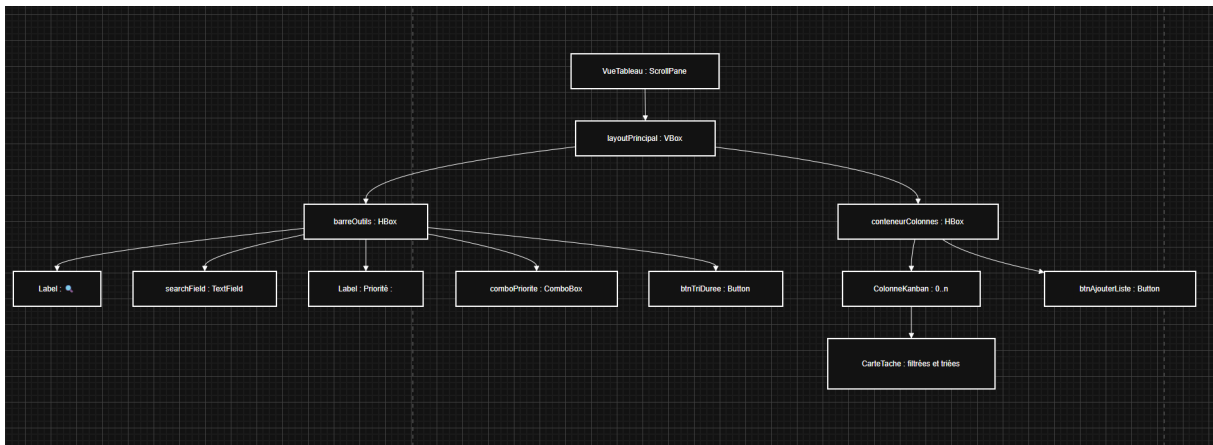


FIGURE 9 – Diagramme de scène de la vue Kanban

8.3 Diagramme de Scène – Vue Liste

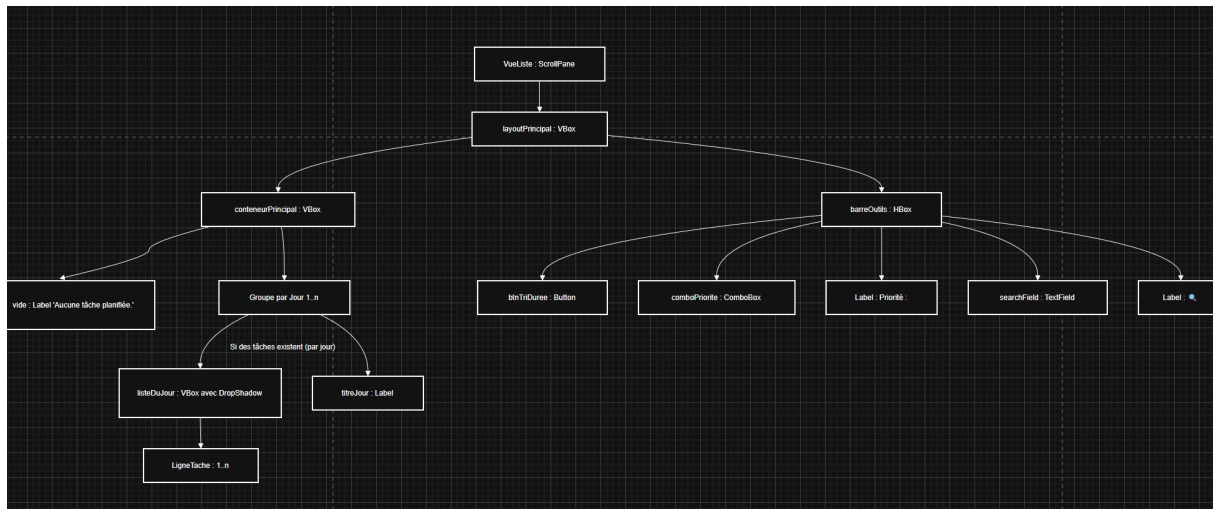


FIGURE 10 – Diagramme de scène de la vue Liste

8.4 Diagramme de Scène – Vue Gantt

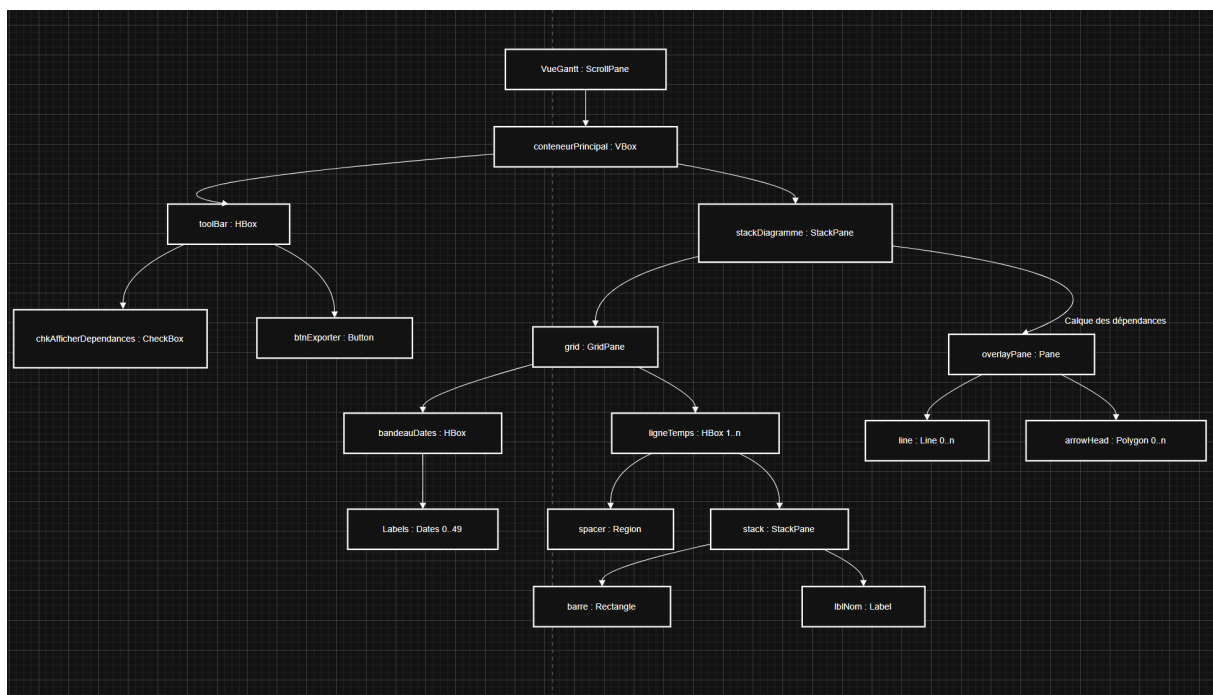


FIGURE 11 – Diagramme de scène de la vue Gantt

8.5 Diagramme de Scène – Vue Archive

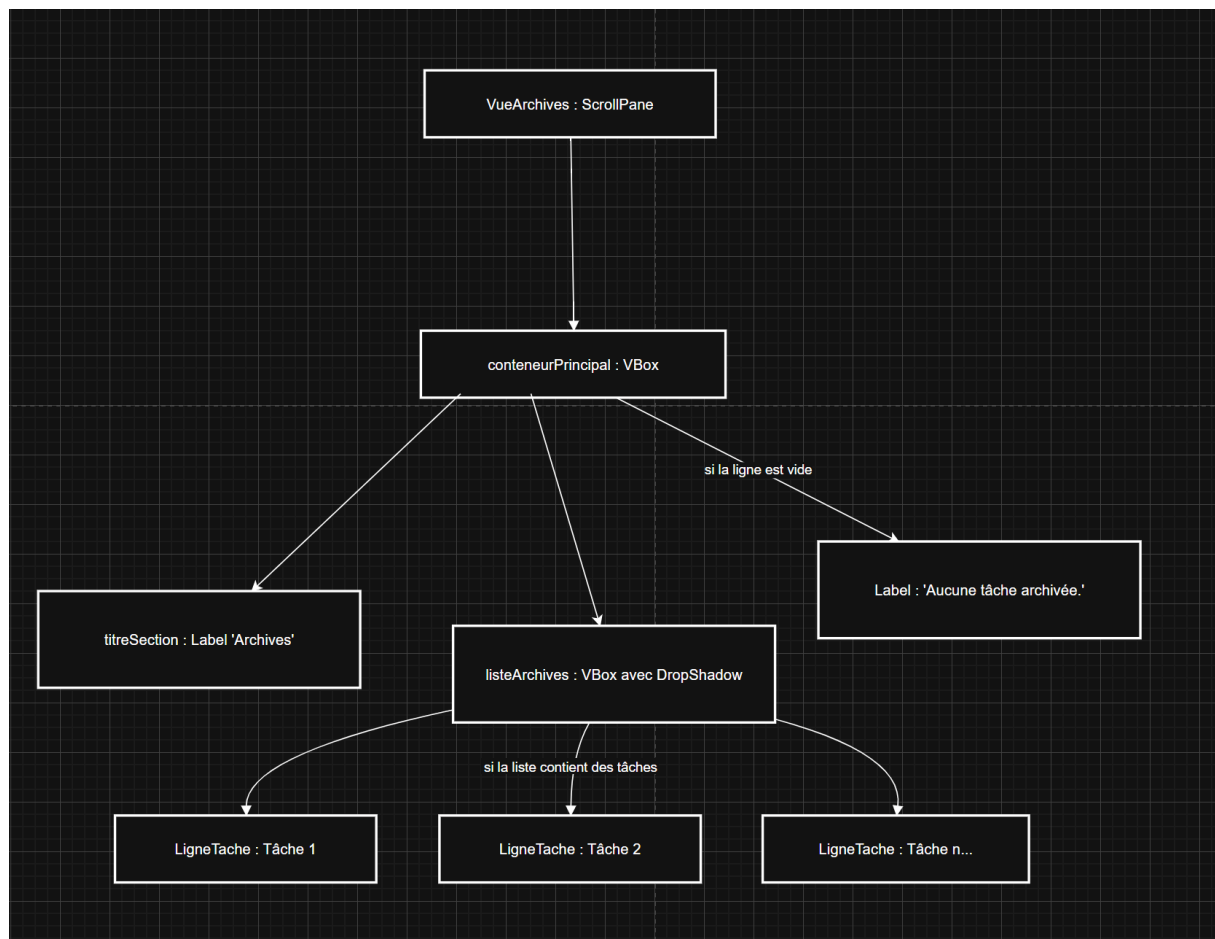


FIGURE 12 – Diagramme de scène de la vue Archive

8.6 Diagramme de Scène – Vue Calendrier

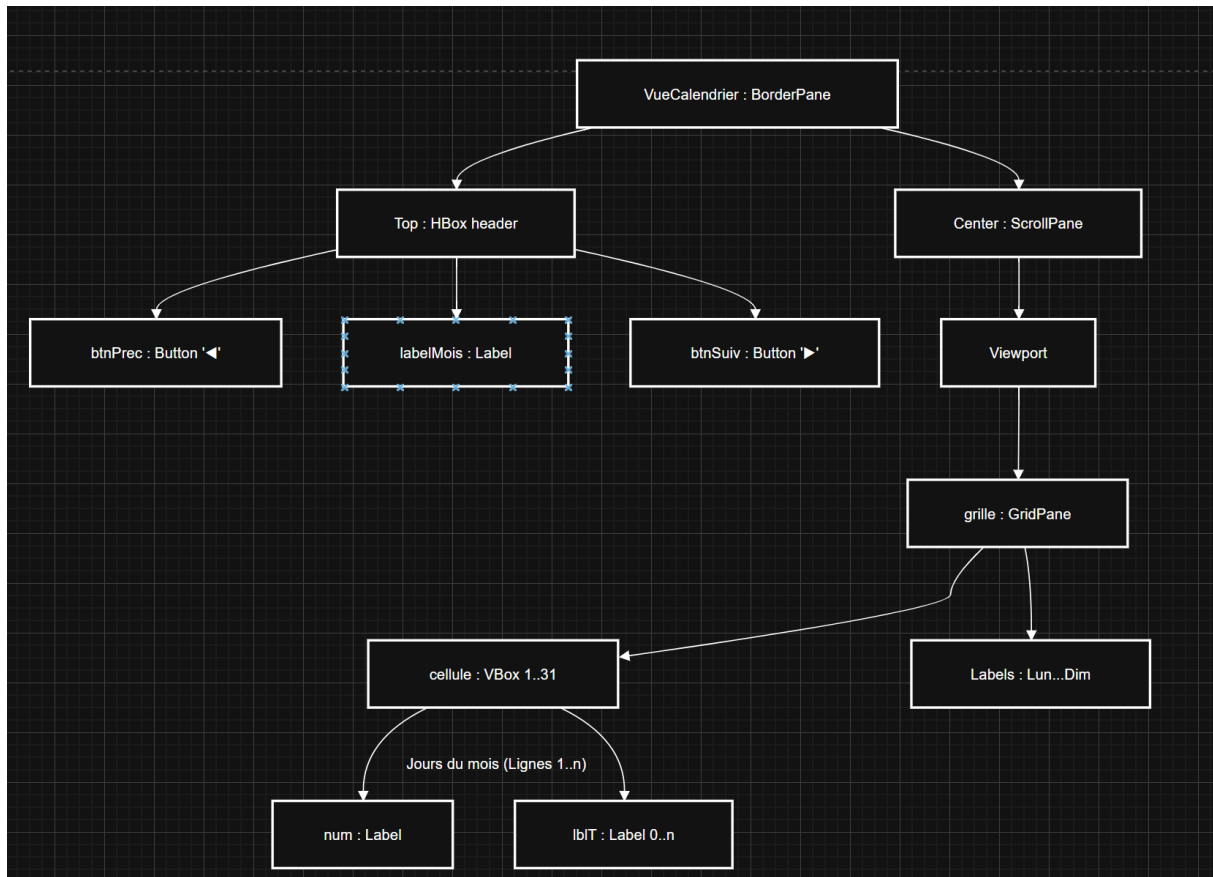


FIGURE 13 – Diagramme de scène de la vue Calendrier

8.7 Organisation des Vues

L'application propose cinq vues distinctes pour visualiser et gérer les projets :

- **Vue Kanban** – Affichage des tâches en colonnes par liste, style tableau Kanban
- **Vue Liste** – Affichage linéaire des tâches sous forme de liste
- **Vue Gantt** – Visualisation temporelle des tâches avec leurs dépendances
- **Vue Archive** – Consultation des tâches archivées
- **Vue Calendrier** – Visualisation des tâches dans un calendrier

8.8 Mécanisme de Changement de Vue

Le changement de vue s'effectue via un menu situé en haut de l'interface, composé d'icônes. Chaque icône est un bouton permettant de basculer vers la vue correspondante. L'utilisateur peut ainsi naviguer librement entre les différentes représentations de ses données en un simple clic.

9 Mode d’Emploi – Lancement sur IntelliJ

Pour les instructions détaillées de lancement de l’application, veuillez consulter le fichier `README.md` fourni avec le projet.

Ce fichier contient :

- Les prérequis (versions de Java, JavaFX)
- Les dépendances du projet
- Les instructions d’installation et de configuration
- L’organisation des fichiers du projet
- Le guide d’accès aux fonctionnalités

10 Conclusion

Ce projet de développement d’application JavaFX nous a permis de mettre en pratique les concepts de programmation orientée objet, les patrons de conception et l’architecture logicielle. L’approche itérative adoptée a facilité la gestion du développement et l’intégration progressive des fonctionnalités.

L’application finale offre une solution complète de gestion de projet en local, avec des fonctionnalités de visualisation variées (Kanban, Gantt, Calendrier) et une gestion avancée des tâches (sous-tâches, dépendances, étiquettes).

11 Liens et Ressources

- **Dépôt GitHub** : https://github.com/bougnouxleo/SAE_3_01_BOUGNOUX_SPATH_MARIQUE
- **Tableau Trello** : <https://trello.com/b/COR9cQFc/sae-31-developpement-dune>

QR Code du Tableau Trello

